

1 Solution — GIAM 1.4

Problem 2

1.1 Problem

Dividing an integer by two has an interesting interpretation in binary: simply shift the digits to the right. Thus $22 = 10110_2$ divided by two gives $1011_2 = 8 + 2 + 1 = 11$. How can you recognize a doubly-even integer from its binary representation?

1.2 The Answer

A doubly-even integer is recognized at a glance:

n is doubly-even $\iff$ its binary representation ends in 00 (the last two bits are both 0).

1.3 Why — From Shifting to Trailing Zeros

Build it up from the problem's own “shift right = divide by two” idea.

Even $\iff$ ends in 0. When you shift a binary number right, the bit that “falls off” the end is the remainder upon dividing by 2. So the division is *exact* — the number is even — exactly when that last bit is 0. (This is the textbook's Theorem 1.4.2: an integer is even iff its units binary digit is 0.)

Doubly-even $\iff$ ends in 00. Recall (Problem 1) that n is doubly-even iff n is even *and* its half $n/2$ is again even. In binary:

- n even means the last bit is 0; shifting right by one gives $n/2$.
- $n/2$ even means *its* last bit is 0 too — i.e. the *second-to-last* bit of the original n is 0.

So both of the bottom two bits of n must be 0: the binary representation ends in 00. Equivalently, since doubly-even = divisible by $4 = 2^2$, and $2^2 = 100_2$, a multiple of 4 must end in two zeros.

1.4 Checking the Example

The problem's own number settles things instantly. $22 = 10110_2$ ends in "10", *not* "00". So **22** is **even** (it ends in a 0) but **not doubly-even**. Indeed, shifting right once gives $1011_2 = 11$, which is *odd* — its half is odd, so **22** fails the second halving. (And $22 = 4 \cdot 5.5$ is not a multiple of 4; $22 \equiv 2 \pmod{4}$.)

1.5 Picture — Read the Last Two Bits

A doubly-even integer ends in 00 in binary (its last two bits are both 0).

n	binary (last two bits boxed)	doubly-even?
4	1 0 0	✓ yes — ends 00
8	1 0 0 0	✓ yes — ends 00
12	1 1 0 0	✓ yes — ends 00
2	1 0	✗ no — ends 10
6	1 1 0	✗ no — ends 10
22	1 0 1 1 0	✗ no — ends 10

2, 6, 22 are even (they end in a single 0) but not doubly-even (they don't end in 00).

Shift-right ($= \div 2$) drops the last bit. Doubly-even $\Leftrightarrow$ the first two dropped bits are both 0:

12: $1100 \rightarrow 110 \rightarrow 11$ (dropped 0, then 0) ✓ doubly-even
 22: $10110 \rightarrow 1011 \rightarrow 101$ (dropped 0, then 1) ✗ not

Figure 1.1: Top: $4 = 100$, $8 = 1000$, $12 = 1100$ all end in 00 (green) — doubly-even. $2 = 10$, $6 = 110$, $22 = 10110$ end in 10 (red) — even but not doubly-even. Bottom: shifting right twice, $12 = 1100$ drops 0 then 0 (doubly-even), while $22 = 10110$ drops 0 then 1 (not).

1.6 Remark — The General Rule: Divisible by $2^k \Leftrightarrow k$ Trailing Zeros

The pattern continues perfectly. Shifting right k times divides by 2^k and discards the bottom k bits; the division is exact iff all k discarded bits were 0. So:

$2^k \mid n \iff$ the binary representation of n ends in (at least) k zeros.

property divisible by binary ends in

even	2	0
doubly-even	4	00
triply-even	8	000
k -times even	2^k	k zeros

Table 1.1

This is the binary mirror of Problem 1’s “ k -times even” hierarchy. The number of **trailing zeros** in the binary expansion of n is exactly the **2-adic valuation** $v_2(n)$ from Problem 1 — binary makes that quantity *visible*, just by counting the zeros at the right-hand end.

1.7 Remark — Why Binary Is the Natural Notation Here

Recognizing divisibility by powers of 2 is *trivial* in binary (count trailing zeros) but fiddly in decimal. The decimal tests are:

- divisible by 2: last decimal digit is 0, 2, 4, 6, 8;
- divisible by 4: the number formed by the **last two** decimal digits is a multiple of 4 (e.g. 1,312 — “12” is a multiple of 4 — so $4 \mid 1312$);
- divisible by 8: the last *three* decimal digits form a multiple of 8.

These work, but require arithmetic on a two- or three-digit tail. In binary the very same facts become “is the last bit 0,” “are the last two bits 0,” “are the last three bits 0” — a pure *look*, no computation. The reason is that $10 = 2 \cdot 5$ carries an *extra* factor of 5, muddying the powers of 2, whereas base 2 is built from powers of 2 alone. (This is the same base-sensitivity that governed terminating decimals back in Section 1.1.)

1.8 Remark — Zero Again

The integer 0 is doubly-even (Problem 1), and the binary rule agrees: $0 = 0_2$ is *all* zeros, so it certainly “ends in 00” — in fact it ends in arbitrarily many zeros, the binary echo of “0 is infinitely even” ($v_2(0) = \infty$). The number of trailing zeros of 0 is unbounded, matching that $2^k \mid 0$ for every k .

1.9 Remark — How Computers Actually Test This

This trailing-bit characterization is not just elegant — it is exactly how real software checks divisibility by powers of **2**. Using a **bitwise AND** with a mask of low **1**-bits:

$$n \text{ is even} \Leftrightarrow (n \& 1) = 0, \quad n \text{ is doubly-even} \Leftrightarrow (n \& 3) = 0,$$

where $3 = 11_2$ masks off the bottom two bits. A processor computes $n \& 3$ in a single instruction — far faster than an actual division — which is why “is this a multiple of **4**?” is one of the cheapest tests a computer can perform. The mathematics of the exercise (“look at the last two bits”) and the engineering of fast code (“mask the last two bits”) are literally the same idea.